Optimization for Neural Networks

Charith Chitraranjan

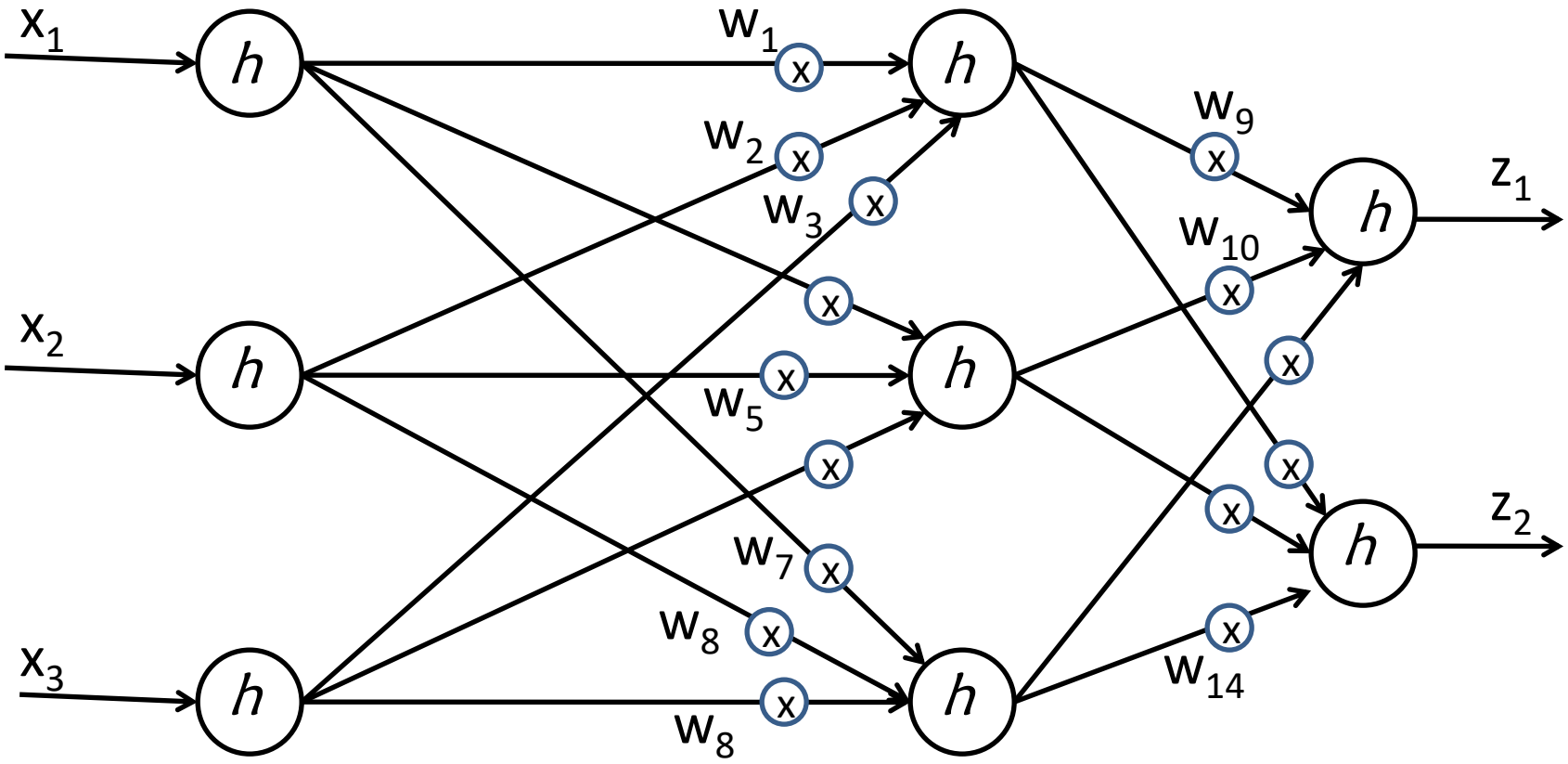


Outline

- Gradient decent
  - With Momentum
  - RMSprop
  - Adam
- Hyperparameter tuning



A Feed Forward Neural Network



- We need to find appropriate values for the weights w_1, …, w_{14} so that the outputs z_1 and z_2 behave desirably.
- Each neuron computes *h(Σwx)*. I.e., *h(weighted sum of its inputs).*
- *h* is some transfer function. E.g., the sigmoid function



Adjusting the Weights

- Consider the same example as before.
- We can compute a prediction error (E) that depends on the mismatch between the desired outputs d^i_1, d^i_2 and predicted outputs z^i_1, z^i_2.
  - E.g., Squared difference

$$E = \sum_{i=1}^{N} [(z_1^i - d_1^i)^2 + (z_2^i - d_2^i)^2]$$

- We can adjust the weights such that E reaches its minimum.

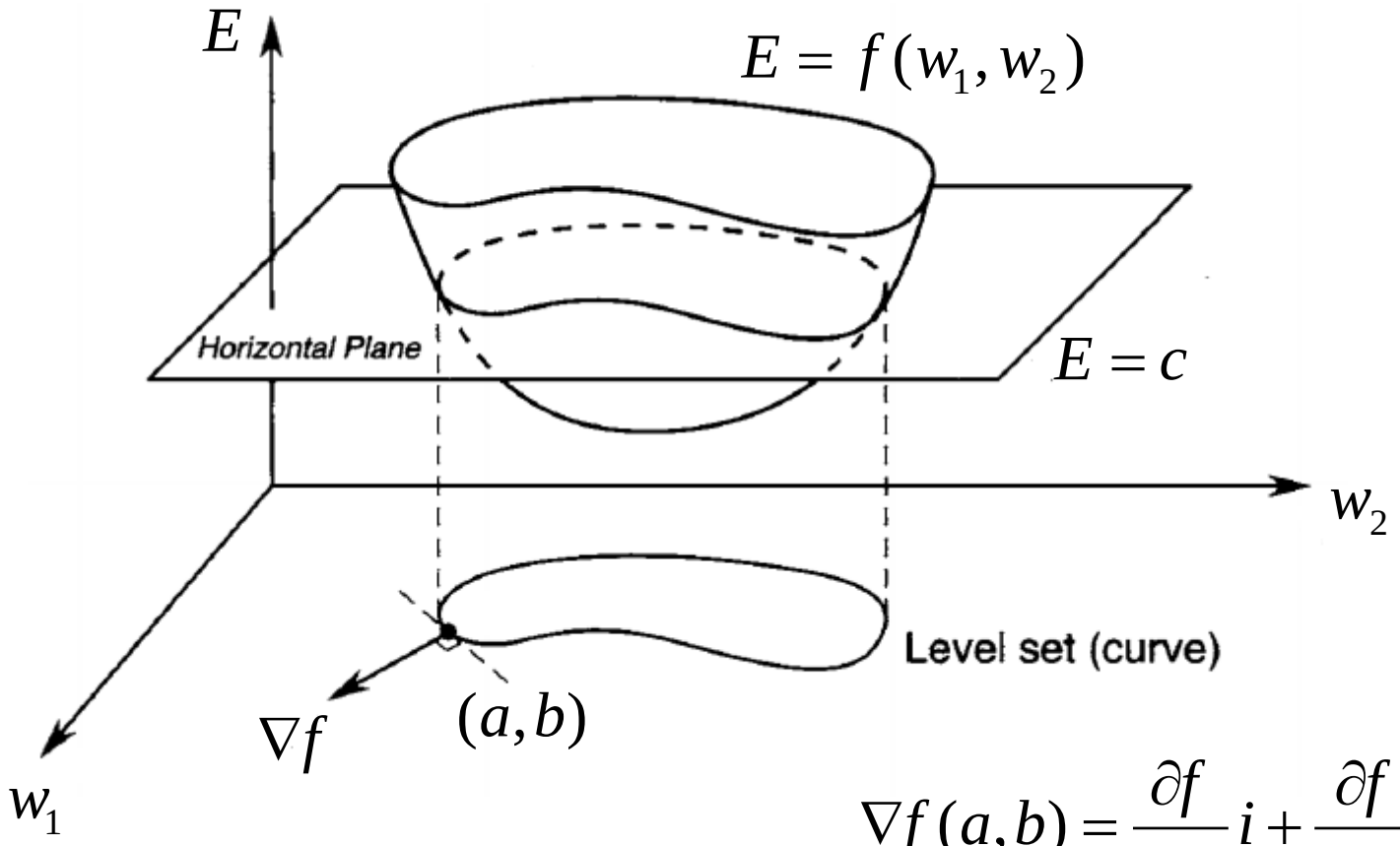


Adjusting the Weights

- Eventually, error (E) is a function of the weights and the inputs.
  - Why not just express E, in terms of w's and x's and minimize it ?
    - For a network of decent size and non-linear transfer functions, expressing E analytically in terms of weights and minimizing it becomes tedious and almost impossible.
    - Neural networks with just input and output layers (no hidden layers) are very limited in what they can do and they need carefully crafted features, which are tedious to get.
    - Neural networks with many hidden layers (deep neural nets), can learn features without much human intervention.
- So, we use numerical methods to adjust the weights.
  - Gradient decent is a popular choice



Gradient Decent

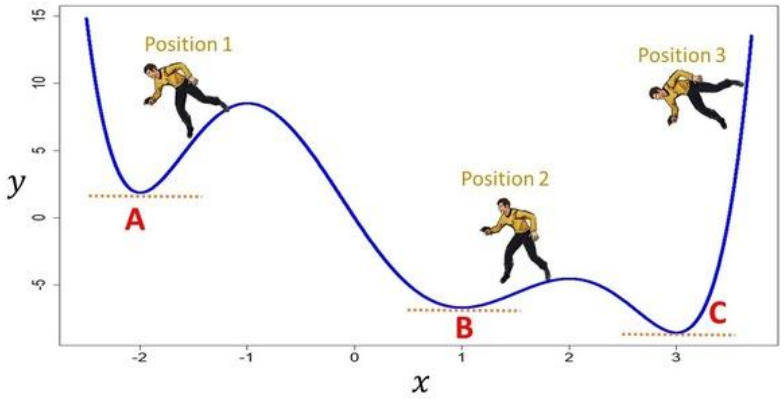


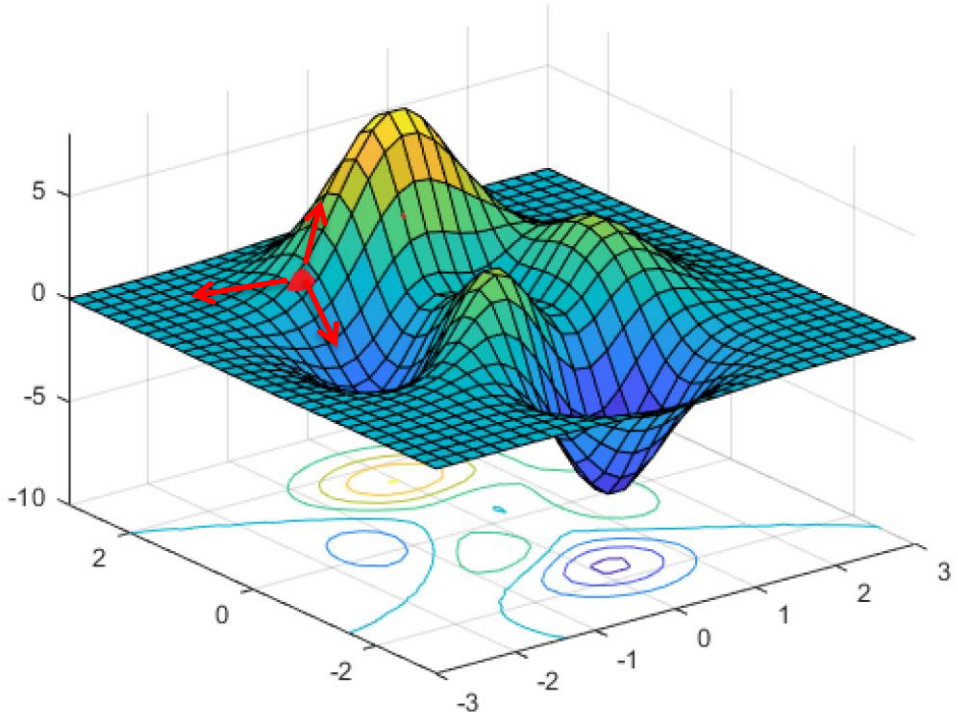
$$\nabla f(a,b) = \frac{\partial f}{\partial w_1} i + \frac{\partial f}{\partial w_2} j, \; at \; (a,b)$$

- Let the current values be w_1=*a* and w_2=*b*.
- We want to reduce the error. So, we need to change w_1 and w_2 along some direction that has a negative gradient (down hill).

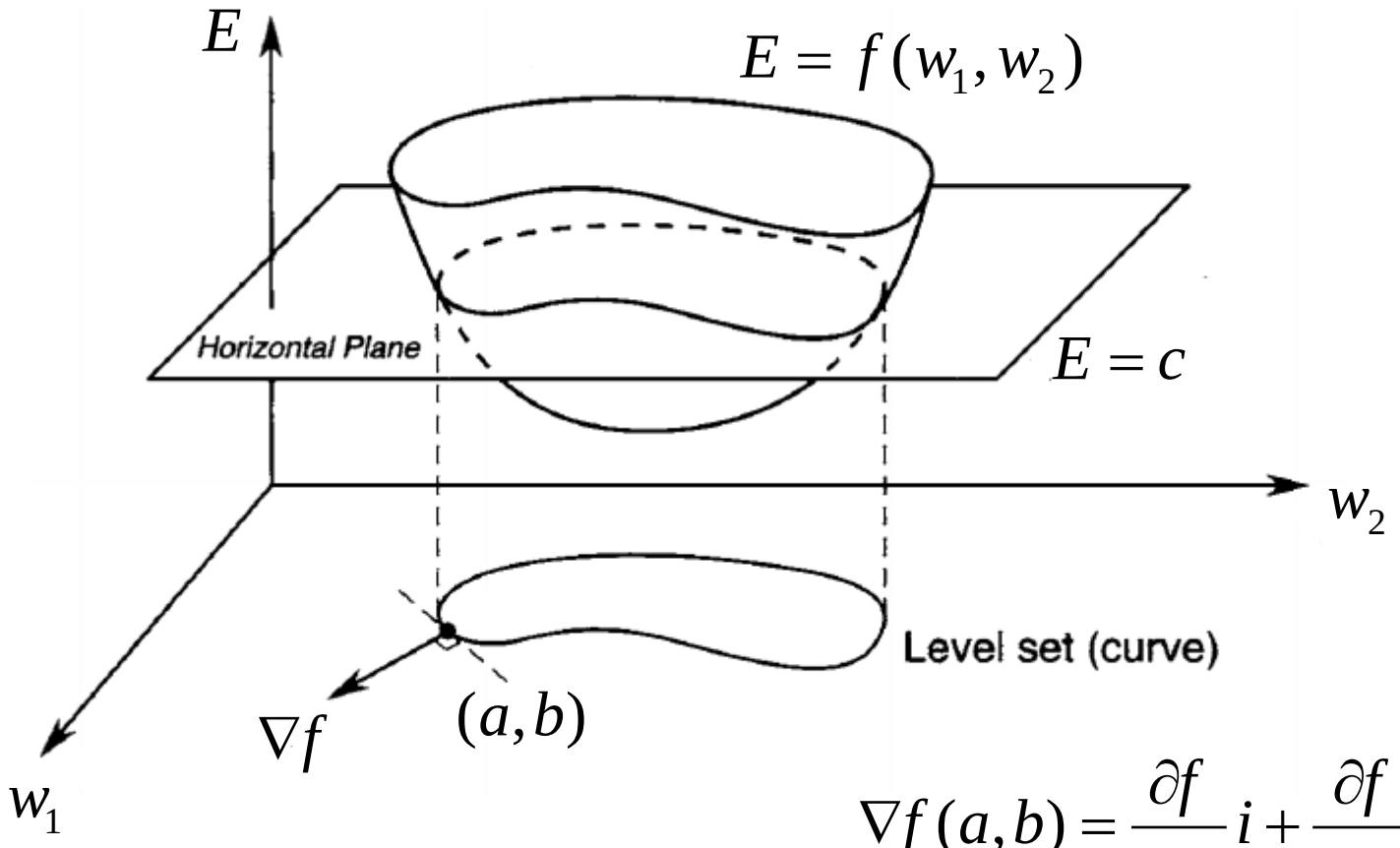


Gradient Decent





Gradient Decent

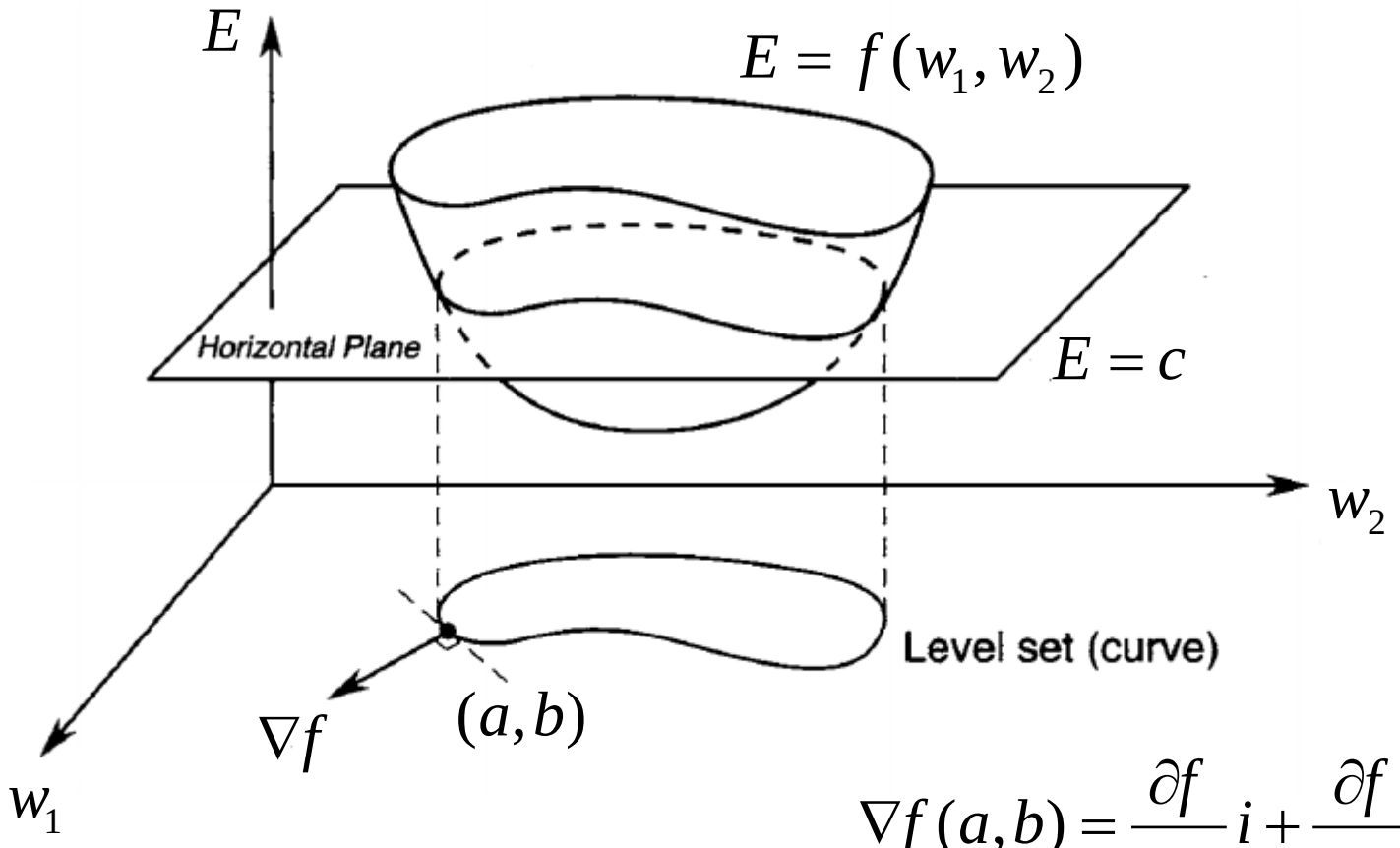


$$\nabla f(a,b) = \frac{\partial f}{\partial w_1} i + \frac{\partial f}{\partial w_2} j, \; at \; (a,b)$$

- Let the current values be w_1=*a* and w_2=*b*.
- We want to reduce the error. So, we need to change w_1 and w_2 along some direction that has a negative gradient (down hill).
- ∇f is the direction of the maximum gradient. So, we move in the opposite direction specified by $-\nabla f$.



Gradient Decent



$$\nabla f(a,b) = \frac{\partial f}{\partial w_1} i + \frac{\partial f}{\partial w_2} j, \; at \; (a,b)$$

- Let the current values be w_1=*a* and w_2=*b*.
- We want to reduce the error. So, we need to change w_1 and w_2 along some direction that has a negative gradient (down hill).
- ∇f is the direction of the maximum gradient. So, we move in the opposite direction specified by $-\nabla f$.



Gradient Decent: Update step

- Let $f(w_1,\ldots w_j,\ldots w_N)$ be a function of N variables.
- Then, in the kth iteration, each variable w_j will be updated as follows.

$$w_j^k = w_j^{k-1} - \alpha \frac{\partial f}{\partial w_j}$$

$$\text{, where } \alpha \text{ is the learning rate and } \frac{\partial f}{\partial w_j} \text{ is the partial derivative}$$

$$\text{of } f \text{ w.r.t } w_j \text{ as evaluated at the point } (w_1^{k-1},\ldots, w_j^{k-1},\ldots, w_N^{k-1})$$



Gradient Decent: Update step

- Let $f(w_1,\ldots w_j,\ldots w_N)$ be a function of N variables.
- Then, in the kth iteration, each variable w_j will be updated as follows

$$w_j^k = w_j^{k-1} - \alpha \frac{\partial f}{\partial w_j}$$

$$\text{, where } \alpha \text{ is the learning rate and } \frac{\partial f}{\partial w_j} \text{ is the partial derivative}$$

$$\text{of } f \text{ w.r.t } w_j \text{ as evaluated at the point } (w_1^{k-1},\ldots, w_j^{k-1},\ldots, w_N^{k-1})$$



Gradient Decent for Learning Parameters in NNs

- For some error function E, the weight update is,

$$w_j^k = w_j^{k-1} - \alpha \frac{\partial E}{\partial w_j}$$

- With the error functions we use (E.g., squared error or cross-entropy), all training data must be explored before the error can be computed.

$$E = \sum_{n=1}^{N} e_n, \text{ where } e_n \text{ is the error for data sample } n$$

- This is called batch gradient decent. However, for large dataset size *N*, this is going to be time consuming.



Stochastic Gradient Decent for Learning Parameters in NNs

- One solution is to update the parameters after exploring each training example (typically, the training examples are randomly ordered).

$$For\ each\ n \in N$$

$$w_j^k = w_j^{k-1} - \alpha \frac{\partial e_n}{\partial w_j}$$

- This usually converges faster to a reasonable solution and is less susceptible to global optima.
- However, it may not converge to the best solution, but hover around it.
  - This is may be good enough in some cases though.



- To have the best of both options, we often use a hybrid method called mini batch gradient decent.

Mini-batch Gradient Decent for Learning Parameters in NNs

- Idea: update the parameters after exploring every *m* samples, where *m*<<*N*.

$$For\ i = m\ to\ N$$

$$E_m = \sum_{u=i-m}^{i} e_u$$

$$w_j^k = w_j^{k-1} - \alpha \frac{\partial E_m}{\partial w_j}$$

$$i \leftarrow i + m$$



Gradient Descent with Momentum

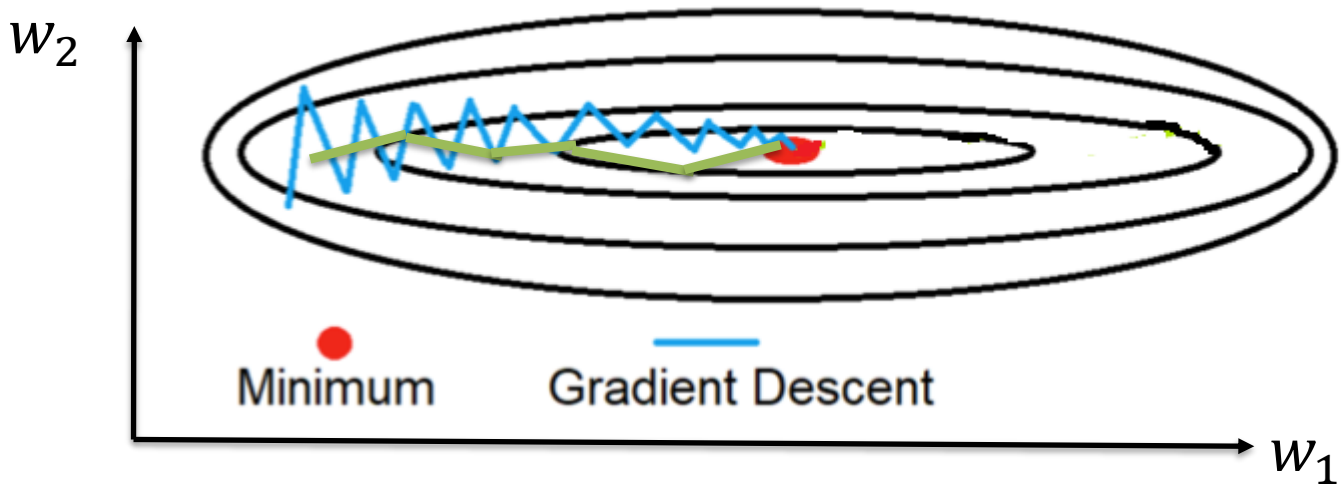


Image adapted from: https://optimization.cbe.cornell.edu/index.php?title=Momentum

- Gradient descent may need many iterations due to oscillations – learning rate will need to be kept small to prevent overshoot
- **Solution:** compute a moving average for the gradient to cancel/smooth out the oscillations.

$$v_j^{(k)} = \beta_1 v_j^{(k-1)} + (1-\beta_1)\frac{\partial E_m}{\partial w_j}$$

$w_j^{(k)} = w_j^{(k-1)} - \alpha v_j^{(k)}$, where α is the learning rate.

- This allows a higher learning rate to enable faster convergence.

Exponential Moving Average (EMA)

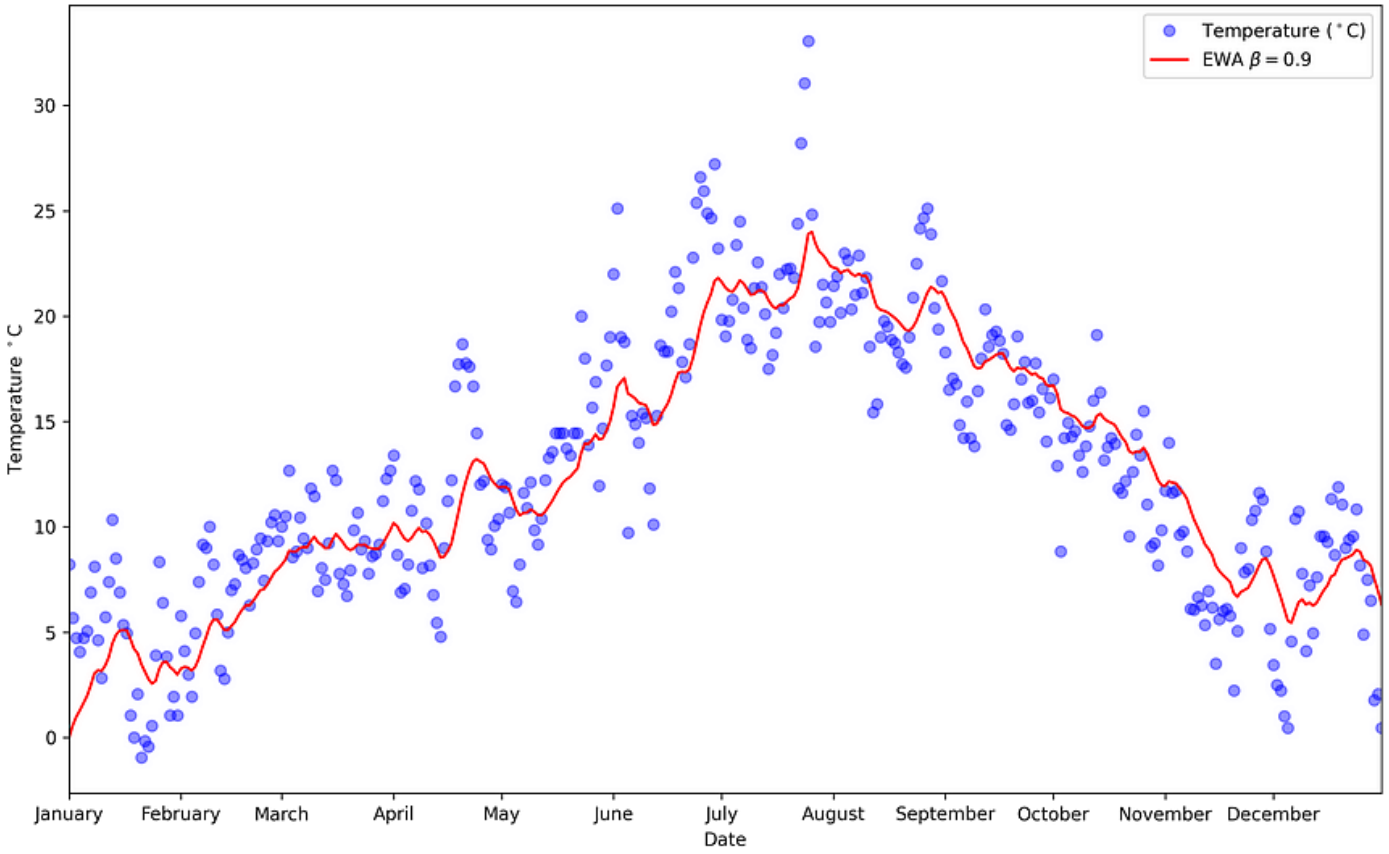


Image source: https://medium.com/mlearning-ai/exponentially-weighted-average-5eed00181a09

$$v^{(n)} = \beta v^{(n-1)} + (1-\beta)T^{(n)}$$
$$v^{(0)} = 0$$
$$v^{(1)} = \beta v^{(0)} + (1-\beta)T^{(1)}$$
$$v^{(2)} = \beta v^{(1)} + (1-\beta)T^{(2)} = \beta(\beta v^{(0)} + (1-\beta)T^{(1)}) + (1-\beta)T^{(2)}$$
$$= \beta^2 v^{(0)} + (1-\beta)T^{(2)} + \beta\,(1-\beta)T^{(1)}$$

Exponential Moving Average (EMA)

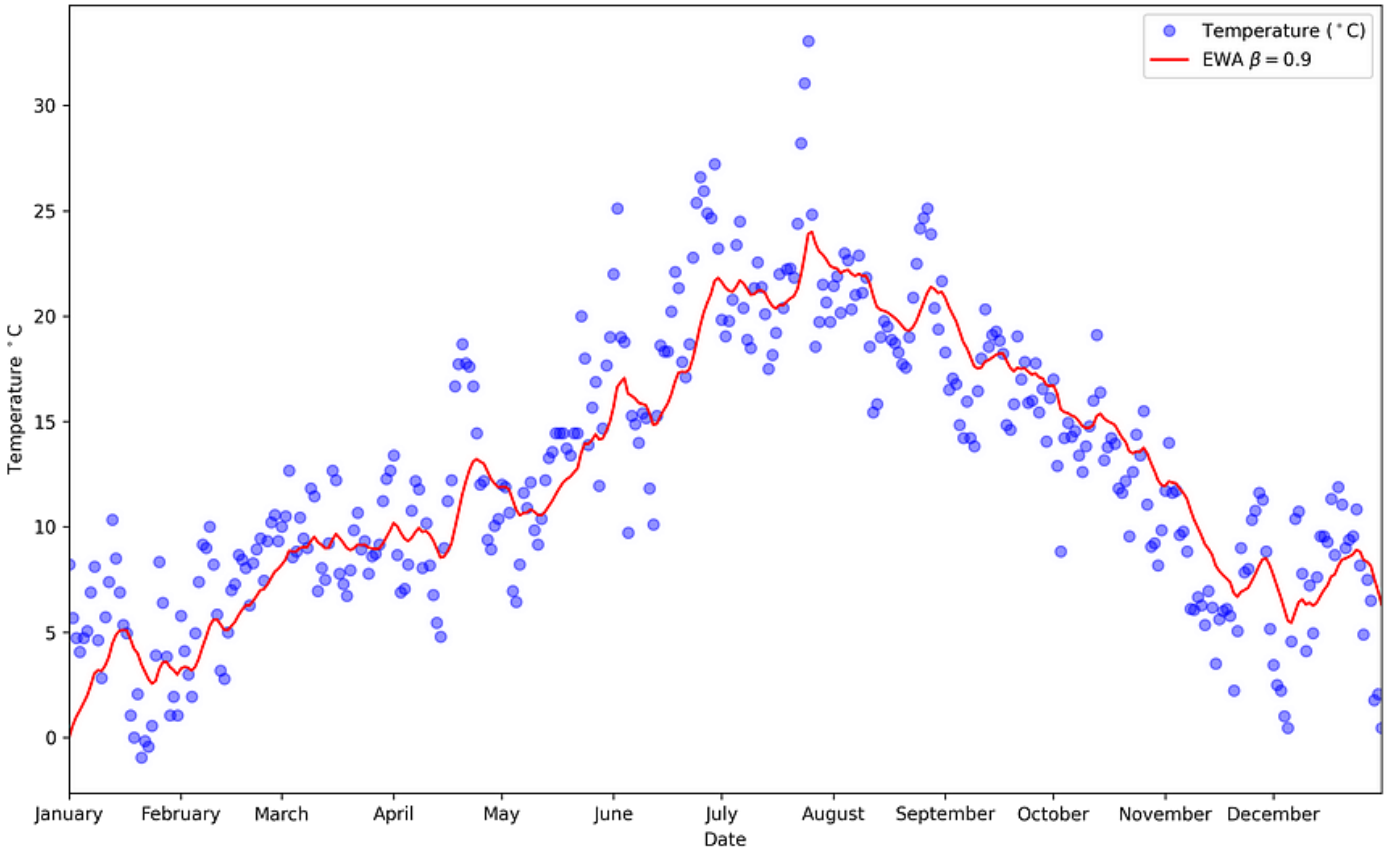


Image source: https://medium.com/m learning-ai/exponentially-weighted-average-5eed00181a09

$$v^{(2)} = \beta v^{(1)} + (1-\beta)T^{(2)} = \beta(\beta v^{(0)} + (1-\beta)T^{(1)}) + (1-\beta)T^{(2)}$$
$$= \beta^2 v^{(0)} + (1-\beta)T^{(2)} + \beta\,(1-\beta)T^{(1)}$$

$$v^{(3)} = \beta v^{(2)} + (1-\beta)T^{(3)} = \beta\left(\beta^2 v^{(0)} + (1-\beta)T^{(2)} + \beta\,(1-\beta)T^{(1)}\right) + (1-\beta)T^{(3)}$$
$$= \beta^3 v^{(0)} + (1-\beta)T^{(3)} + \beta(1-\beta)T^{(2)} + \beta^2(1-\beta)T^{(1)}$$

Exponential Moving Average (EMA)

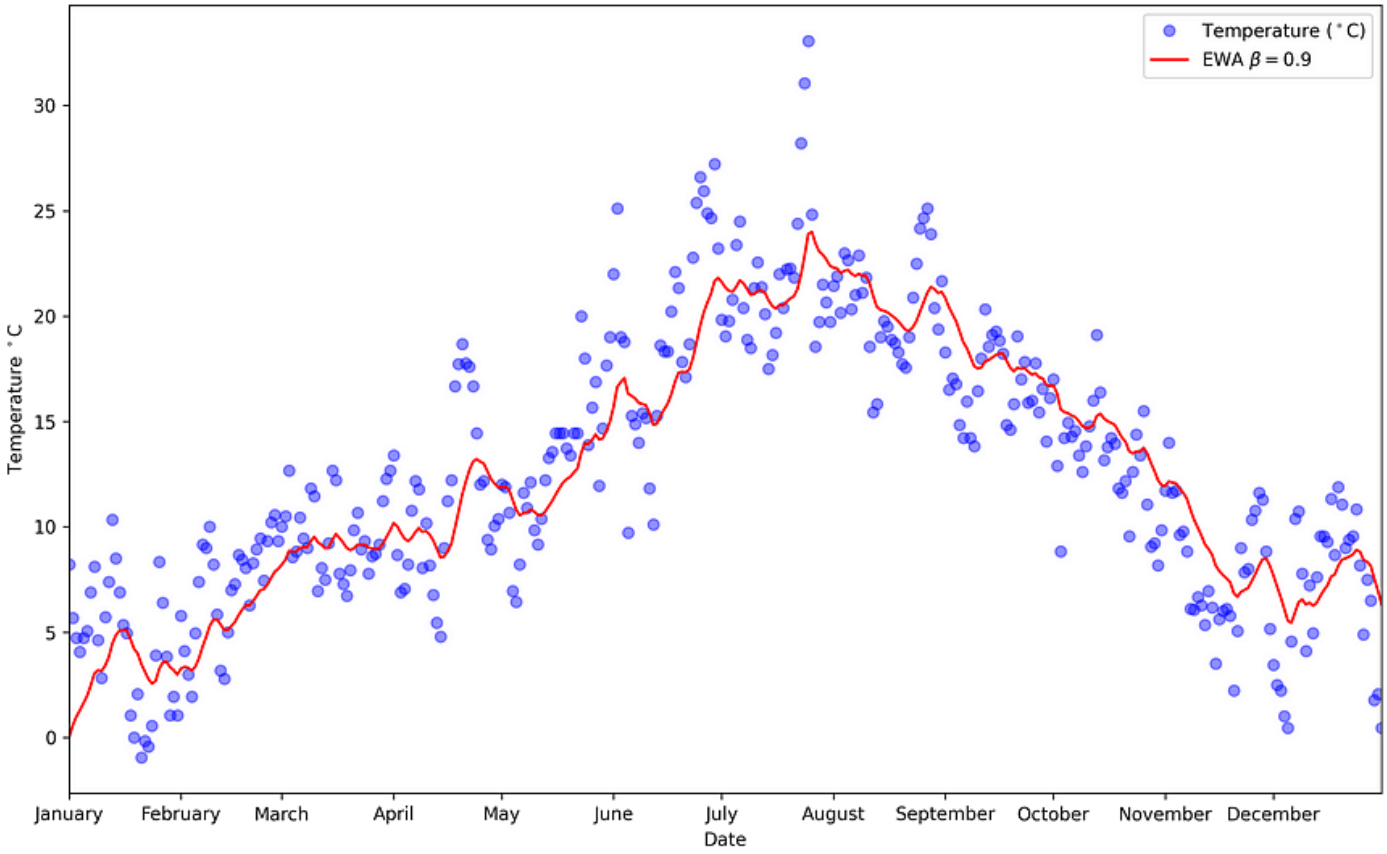


Image source: https://medium.com/m learning-ai/exponentially-weighted-average-5eed00181a09

$$v^{(n)} = \beta v^{(n-1)} + (1-\beta)T^{(n)}$$
$$= \beta^n v^{(0)} + (1-\beta)T^{(n)} + \beta(1-\beta)T^{(n-1)} + \beta^2(1-\beta)T^{(n-2)} + \cdots + \beta^{n-1}(1-\beta)T^{(1)}$$

Exponential Moving Average (EMA)

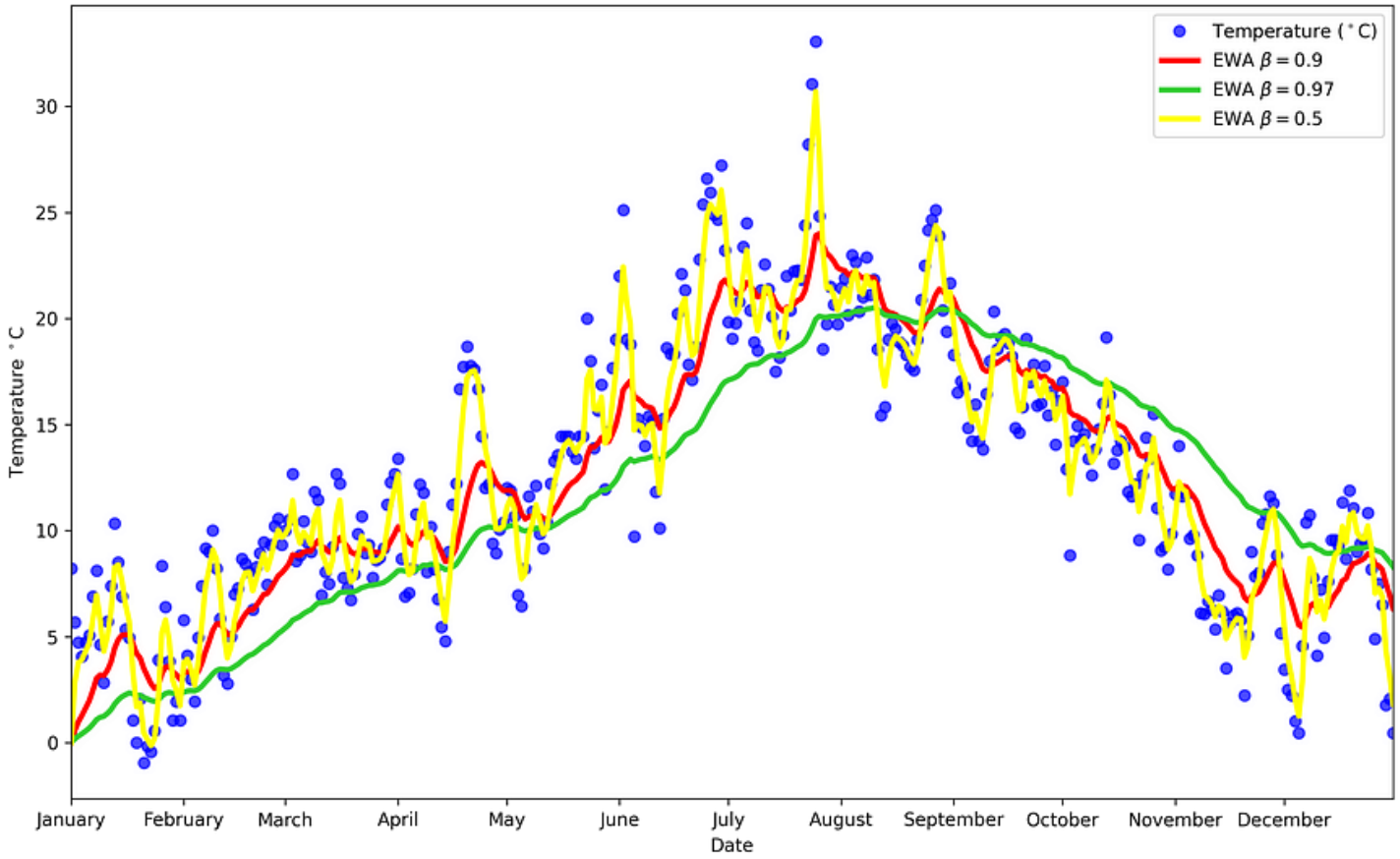


Image source: https://medium.com/m learning-ai/exponentially-weighted-average-5eed00181a09

$$v^{(n)} = \beta v^{(n-1)} + (1-\beta)T^{(n)}$$
$$= \beta^n v^{(0)} + (1-\beta)T^{(n)} + \beta(1-\beta)T^{(n-1)} + \beta^2(1-\beta)T^{(n-2)} + \cdots + \beta^{n-1}(1-\beta)T^{(1)}$$

Approximately, $\frac{1}{(1-\beta)}$ number of values in the past are averaged.

EMA: Bias correction

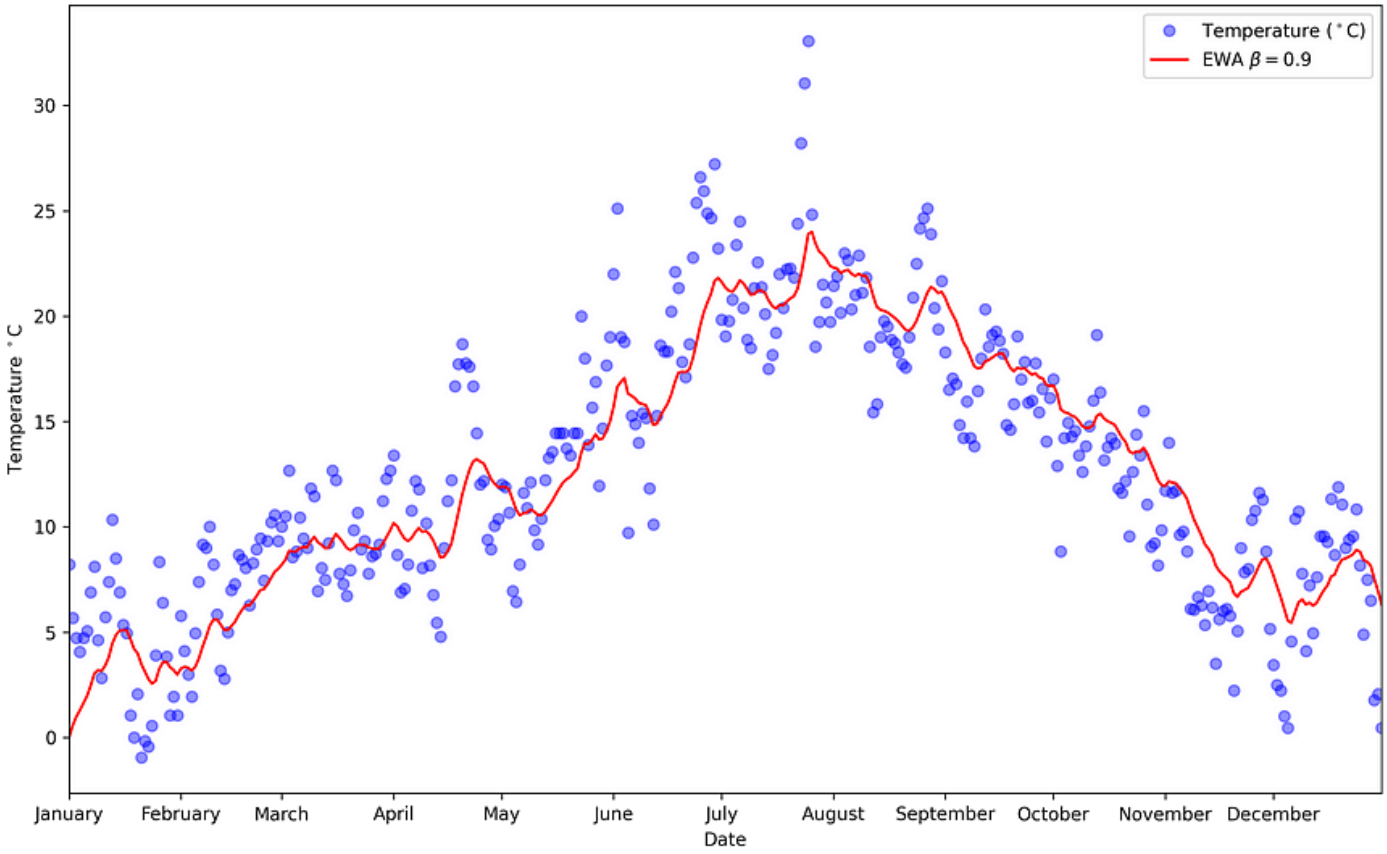


Image source: https://medium.com/m learning-ai/exponentially-weighted-average-5eed00181a09

$v^{(n)} = \beta v^{(n-1)} + (1-\beta)T^{(n)}$

$v^{(0)} = 0$

$v^{(1)} = \beta v^{(0)} + (1-\beta)T^{(1)}$

For $\beta = 0.9$,

$v^{(1)} = 0.9 \times 0 + 0.1 \times T^{(1)}$, small value

Bias correction: $\frac{v^{(n)}}{1-\beta^n}$

Gradient Descent with Momentum

- Gradient descent with bias corrected momentum

$$v_j^{(k)} = \beta_1 v_j^{(k-1)} + (1-\beta_1)\frac{\partial E_m}{\partial w_j}$$

$$w_j^{(k)} = w_j^{(k-1)} - \alpha \frac{v_j^{(k)}}{1-{\beta_1}^k},$$ where α is the learning rate.



RMSprop

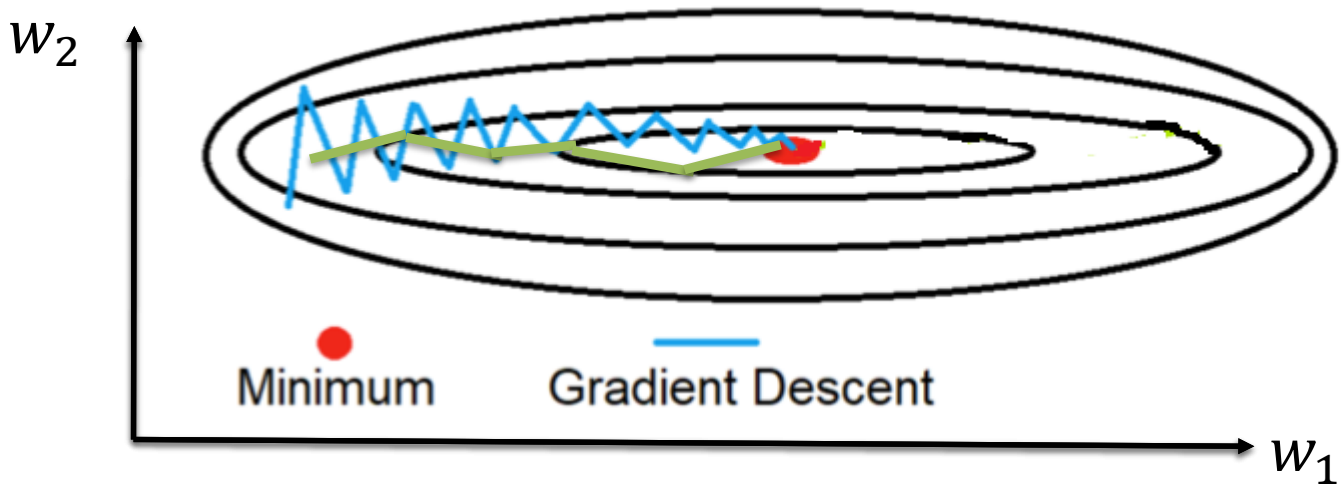


Image adapted from: https://optimization.cbe.cornell.edu/index.php?title=Momentum

- Compute the EMA of the squares of the gradients

$$s_j^{(k)} = \beta_2 s_j^{(k-1)} + (1-\beta_2)\left(\frac{\partial E_m}{\partial w_j}\right)^2$$

$$w_j^{(k)} = w_j^{(k-1)} - \frac{\alpha}{\sqrt{\frac{s_j^{(k)}}{\left(1-\beta_2{}^{k}\right)}+\varepsilon}}\frac{\partial E_m}{\partial w_j}$$, where α is the learning rate.

- This too allows higher learning rate to enable faster convergence.

Adam Optimizer

- Adam – Adaptive moment estimation
- Combines both momentum and RMSprop

$$w_j^{(k)} = w_j^{(k-1)} - \frac{\alpha}{\sqrt{\frac{s_j^{(k)}}{\left(1-\beta_2{}^k\right)}+\varepsilon}} \frac{v_j^{(k)}}{\left(1-\beta_1{}^k\right)},$$

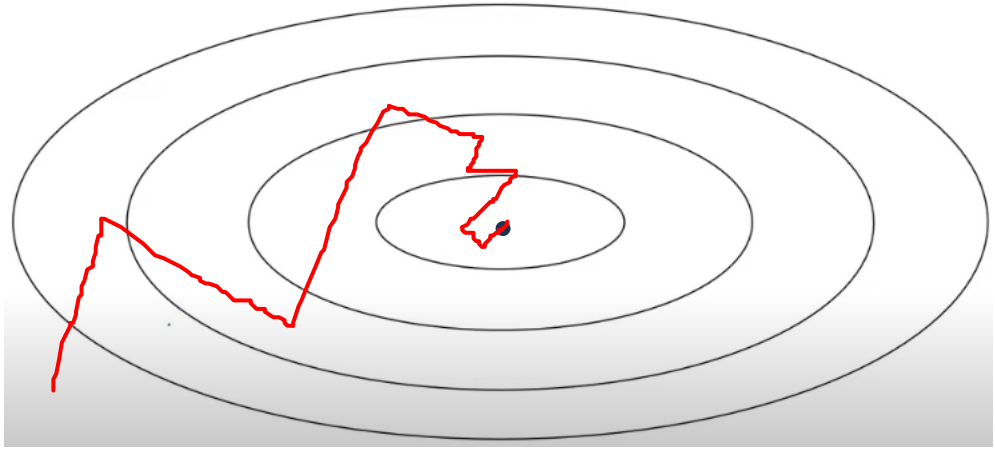
where α is the learning rate.

Default values: $\beta_1 = 0.9, \beta_2 = 0.99, \varepsilon = 10^{-8}$



- Adam enables faster training.

Learning Rate Decay



- Idea: Reduce the learning rate as epochs progress.
  - Helps to move through the search space fast initially and then fine tune towards the (local) optimum.
- Possible options:

$$\alpha = \frac{\alpha_0}{(1 + d_{const} \times ephoc\#)}$$

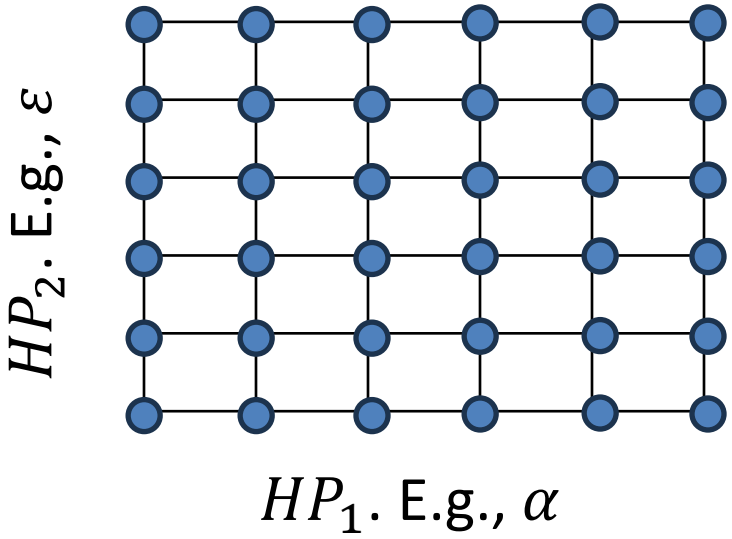
$$\alpha = \lambda^{ephoc\#}\alpha_0, \text{ where } 0 < \lambda < 1.$$

Hyperparameter Tuning

- Potential hyperparameters to tune
  - Learning rate (α).
  - Parameters of Adam (β_1, β_2, and ε) - usually default values are used for these.
  - Mini batch size.
  - Number of layers and number of neurons in each of them.
  - Learning rate decay parameters.
  - Etc.

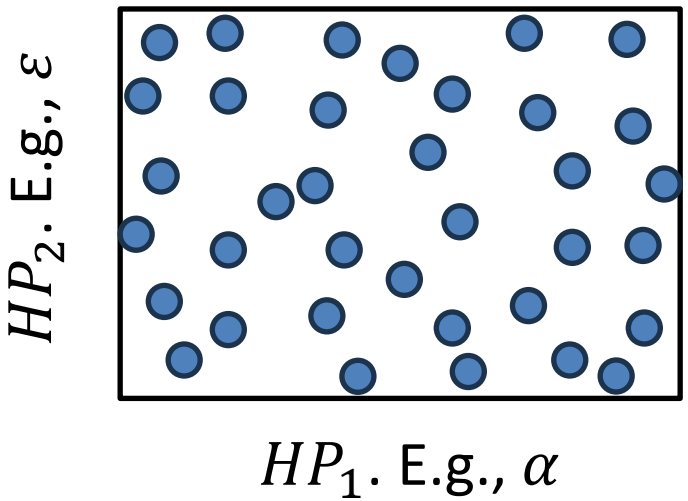


Hyperparameter Tuning: Grid Vs. Random Search



Grid search

- If one or more parameters has little effect on the performance, this can be waste of time.

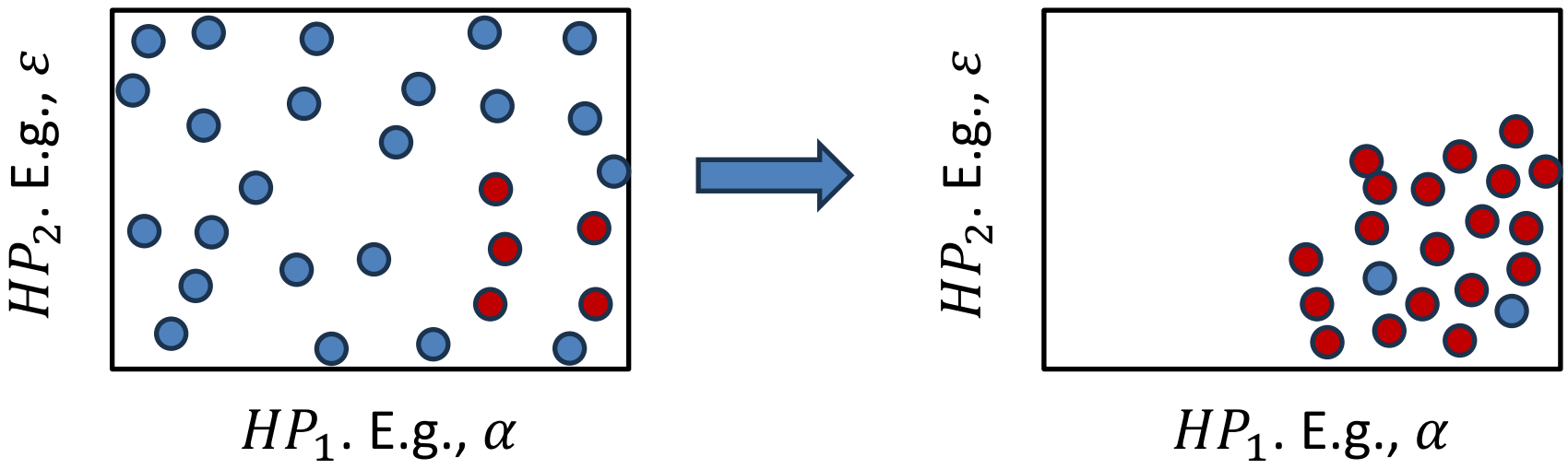


Random search

- Attempts a variety of values for each parameter.

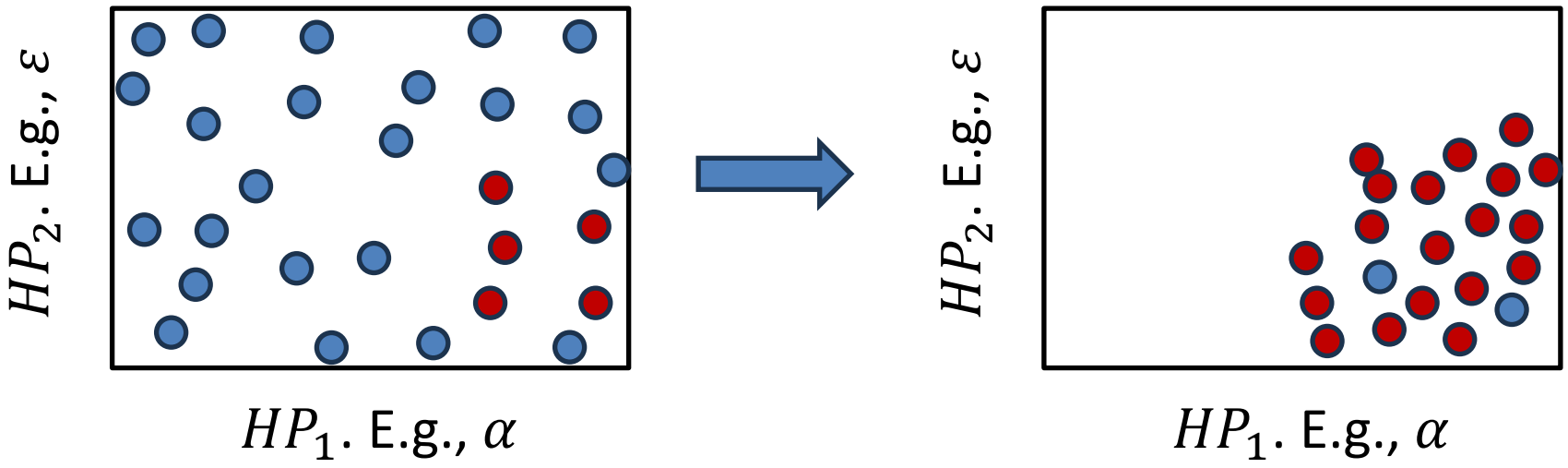


Hyperparameter Tuning: Coarse to Fine Granularity



- Initially, search a larger area coarsely.
- Identify areas that produced better results.
- Do a more fine-grained search in those areas.

Hyperparameter Tuning: Coarse to Fine Granularity



- Use the training dataset or a validation dataset to assess model performance during hyperparameter tuning – leave the test data out of it.

References

- https://www.youtube.com/playlist?list=PLkDaE6sCZn6Hn0vK8co82zjQtt3T2Nkqc
- https://optimization.cbe.cornell.edu/index.php?title=Momentum
- https://optimization.cbe.cornell.edu/index.php?title=RMSProp
- https://www.cs.cmu.edu/~10606/gradient.pdf

